Nom, Prénom : *Baron Julien*
Année académique : *2025-2026*
Intitulé du cours : *Programmation Web II*
Nom de l'enseignant : *J. Riggio*
Titre du projet : HELB Agent — Plateforme d'agents IA et chat collaboratif

Table des matières

1. Introduction
2. Fonctionnalités
3. Analyse
4. Limitations
5. Conclusion
6. Vidéo



1.Introduction

Dans le cadre du cours de développement Web (Q3), une application Django a été développée afin de proposer une plateforme centrée sur des agents d'intelligence artificielle et des interactions de type chat. L'objectif initial consistait à permettre la création et la consultation d'agents IA publics, ainsi que la possibilité pour un utilisateur de converser avec un agent via une interface web. Progressivement, des fonctionnalités supplémentaires ont été ajoutées : intégration d'un modèle IA local via Ollama, gestion complète des agents (CRUD), ajout de statistiques d'utilisation visibles dans le profil, et enfin intégration d'une page HELBPlays affichant des ressources externes (chat.txt et frame.jpg).

Ce rapport est structuré de manière à présenter, d'abord, les fonctionnalités réellement implémentées et testées, ensuite l'analyse technique accompagnée de schémas, puis une section "Limitations" proposant une autocritique orientée sur les contraintes et faiblesses des choix actuels. Enfin, une conclusion récapitule le travail effectué et les pistes d'amélioration réalistes.

2.Fonctionnalités

Gestion d'utilisateurs et profils

Un système d'authentification standard Django a été mis en place afin de distinguer les utilisateurs connectés des visiteurs. La création d'un compte et l'authentification ont été rendues nécessaires pour accéder aux fonctionnalités principales du site, notamment la création d'agents et l'utilisation du chat avec IA. Une fois connecté, l'utilisateur dispose d'une page de profil permettant l'édition des informations principales (username, email) ainsi que l'upload d'une image de profil. Ce profil a été utilisé comme point central d'affichage des statistiques d'activité afin d'éviter une page séparée "stats" et de conserver une logique simple : les informations personnelles et l'activité sont regroupées au même endroit.

En complément du profil "privé" (modifiable), un profil "public" a été mis en place pour permettre l'accès au profil d'un autre utilisateur depuis l'interface de chat. Cette séparation a permis d'éviter que les champs de modification ne soient visibles lorsqu'un autre utilisateur est consulté. Dans cette logique, certaines statistiques ont été volontairement restreintes : l'activité "messages par jour" est considérée comme une information personnelle et n'est affichée que pour l'utilisateur lui-même.

Agents IA publics

Les agents IA ont été définis comme des entités publiques. Un agent est caractérisé par un nom, une description, un type de modèle (local vs API), un nom de modèle (ex. smollm:135m), un "system prompt" (rôle/consignes de comportement), un délai "cooldown" (contrainte de fréquence d'envoi), et éventuellement un avatar. Conformément aux consignes récentes, le système a été ajusté pour ne pas proposer d'agents privés : l'affichage des agents et la navigation ont été pensés autour de la publication et de la consultation d'agents accessibles à tous.

L'affichage des agents est assuré via :

- une page liste présentant les agents disponibles ;
- une page détail affichant les caractéristiques de l'agent (description, modèle, prompt, cooldown, etc.) ;
- une action "discuter" menant directement à l'interface de chat liée à cet agent.

Bien que des fonctions de gestion "Mes agents" aient été mises en place pendant une itération (CRUD), les consignes actuelles imposant des agents publics ont entraîné une révision : l'interface est orientée vers la consultation publique, et les éléments relevant d'une logique "privée" doivent être retirés ou neutralisés selon l'interprétation stricte de l'énoncé final.

Chat avec agent IA

Une interface de chat a été implémentée pour permettre une conversation structurée entre un utilisateur et un agent sélectionné. Chaque message envoyé via le formulaire est persisté dans la base de données, ce qui garantit qu'un historique de discussion est visible au rechargement de la page. L'affichage du chat a été conçu pour différencier clairement les messages "utilisateur" et les messages "agent", notamment via un booléen is_user.

Une amélioration importante a été ajoutée sur l'ergonomie : dans la fenêtre de chat, les noms affichés sont cliquables. Ainsi :

- le nom de l'utilisateur dans un message mène vers son profil public ;
- le nom de l'agent mène vers la page détail de cet agent.

Cette navigation améliore la compréhension du contexte, notamment quand plusieurs utilisateurs testent des agents différents. Par ailleurs, un souci d'affichage doublé du chat a été corrigé : un bloc HTML provenant de base.html injectait des messages au-dessus de la zone principale, ce qui donnait l'impression que l'interface était répétée. Après suppression/ajustement de ce bloc, une seule zone de chat cohérente a été conservée.

Intégration IA locale (Ollama + smollm:135m)

L'IA locale a été intégrée via Ollama. L'objectif était de permettre des réponses générées localement, sans dépendre d'un service externe. Le modèle smollm:135m a été utilisé comme modèle de démonstration.

Une difficulté technique centrale a été rencontrée : l'API d'Ollama peut renvoyer des réponses en streaming (plusieurs objets JSON successifs).

Enfin, afin d'améliorer la pertinence des réponses (langue et longueur), le "system prompt" a été renforcé et rendu plus direct. Les petits modèles ont tendance à "dérailler" quand les consignes sont trop longues ; une formulation courte, répétée et explicite ("répondre en français", "1–2 phrases maximum") a montré un comportement plus stable.

Statistiques dans la page profil (Chart.js)

Un module de statistiques a été intégré directement dans la page profil. L'objectif était de transformer l'historique des messages stockés en indicateurs visibles et facilement interprétables. Deux graphiques ont été mis en place :

1. Messages par agent (bar chart) : permet d'identifier avec quels agents l'utilisateur interagit le plus.

2. Messages par jour (line chart) : permet de visualiser l'activité dans le temps (rythme de travail, fréquence d'utilisation).

Sur le plan technique, ces statistiques reposent sur des agrégations Django (Count) et une transformation de date (TruncDate). Elles sont ensuite injectées dans le template et consommées par Chart.js. Une difficulté de contexte a été rencontrée : si les statistiques n'étaient calculées que dans une branche GET, une requête POST (mise à jour du profil) pouvait casser l'affichage. La solution consistait à calculer les variables nécessaires avant le bloc if request.method, garantissant la présence des données dans tous les cas.

HELBPlays (ressources externes + chat collaboratif)

Conformément au nouvel énoncé, une page HELBPlays a été ajoutée afin d'intégrer deux ressources externes amenées à évoluer :

- un fichier texte de chat (chat.txt),
- une image (frame.jpg).

Le rafraîchissement dynamique a été pris en compte : le contenu doit se mettre à jour sans recharger manuellement toute la page. Le principe consiste à recharger régulièrement (polling) le contenu du chat et de l'image. L'envoi de messages vers le chat collaboratif est effectué via une requête GET vers chat.php, en encodant en Base64 le nom d'utilisateur, le message et la clé personnelle.

Cette fonctionnalité a été volontairement séparée du chat des agents : l'énoncé précise qu'il n'est pas demandé que les agents interagissent avec le chat collaboratif. La page HELBPlays constitue donc un module "collectif", distinct de HELBAgent.

3.Analyse

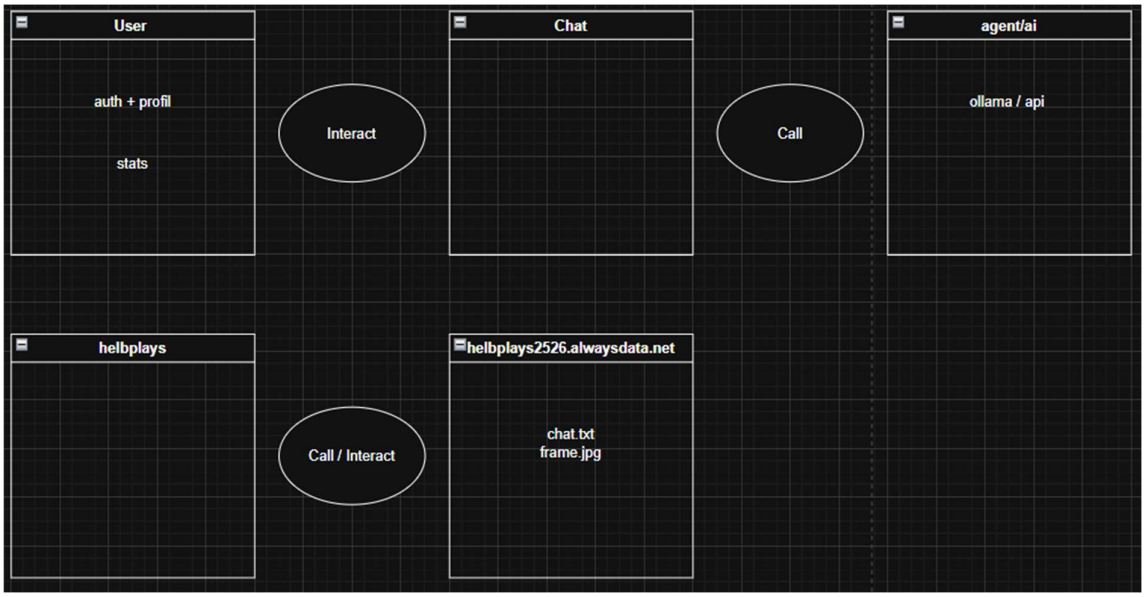
Architecture générale

L'application est structurée selon une architecture Django classique, en séparant les responsabilités :

- agents : définition et affichage des agents IA ;
- chat : stockage et affichage des messages, logique de conversation ;
- users : authentification et profils (privé/public) ;
- helbplays : intégration des ressources externes et envoi de messages au chat collaboratif.

Ce découpage facilite la maintenance : chaque module reste lisible et testable isolément. Les dépendances sont principalement orientées dans un seul sens : chat dépend de agents (un message est lié à un agent), tandis que agents ne dépend pas de chat.

Diagramme 1 — Diagramme de composants



Ce schéma met en évidence deux flux externes :

- vers Ollama en local ;
- vers helbplays2526.alwaysdata.net et l'api en distant.

Le stockage dans la base repose sur des modèles Django. Les éléments clés sont :

- Agent : configuration de l'agent IA ;
- Message : historique de conversation utilisateur/agent ;

- Profile: information d’utilisateur.

La relation Chat → Agent garantit qu’un historique est consultable par agent. La relation Chat → User garantit qu’un historique est consultable par utilisateur. Ce choix est motivé par deux besoins : l’affichage du chat et le calcul des statistiques.

Flux d’une conversation (utilisateur ↔ agent)

Lorsqu’un message est envoyé dans le chat :

1. le message utilisateur est validé;
2. il est enregistré en base;
3. une réponse est générée par l’IA (fonction centralisée dans agents/ai.py) ;
4. la réponse est enregistrée;
5. l’utilisateur est redirigé sur la même page afin de revoir l’historique mis à jour.

La page HELBPlays repose sur deux mécanismes :

- affichage d’un texte (chat.txt) + une image (frame.jpg) ;
- rafraîchissement périodique (polling) côté front-end ;
- envoi d’un message via URL chat.php avec Base64.

Le choix du polling est motivé par la simplicité : un intervalle (ex. toutes les 2 secondes) suffit pour simuler du “temps réel” sans mettre en place WebSockets. Le coût est acceptable pour un petit nombre d’utilisateurs en classe, et le code reste lisible.

4. Limitations

Malgré les fonctionnalités implémentées et le bon fonctionnement global de l'application, plusieurs limites peuvent être identifiées, tant sur le plan technique que fonctionnel. Cette section propose une analyse critique des contraintes actuelles du projet ainsi que des axes d'amélioration envisageables dans un contexte de développement futur.

Tout d'abord, l'application repose sur un serveur Django unique exécuté en environnement de développement. Dans un contexte d'utilisation réelle avec un grand nombre d'utilisateurs simultanés, cette architecture pourrait rapidement montrer ses limites. En particulier, la gestion des requêtes simultanées liées aux discussions avec les agents ou au chat collaboratif HELBPlays pourrait entraîner des ralentissements, voire des indisponibilités temporaires du service. L'absence de mécanismes de mise en cache ou de répartition de charge constitue donc une contrainte importante en termes de scalabilité.

Ensuite, l'intégration de l'intelligence artificielle locale via Ollama repose sur un service externe exécuté sur la machine hôte. Cette dépendance implique que l'application ne fonctionne correctement que si le serveur Ollama est actif et accessible. En cas d'arrêt du service, de surcharge système ou de problème de configuration réseau, les agents deviennent incapables de répondre aux messages des utilisateurs. De plus, les performances et la qualité des réponses sont directement liées aux capacités matérielles de la machine hébergeant le modèle, ce qui peut provoquer des temps de réponse élevés sur des configurations modestes.

Par ailleurs, ce mécanisme reste sensible aux comportements inattendus. Par exemple, un rafraîchissement rapide de la page ou des requêtes multiples envoyées quasi simultanément peuvent conduire à des incohérences visibles dans l'interface utilisateur. Cela montre les limites d'un contrôle principalement réalisé côté serveur sans gestion fine des états de conversation.

Concernant le chat collaboratif HELBPlays, l'application dépend entièrement de ressources externes fournies par un serveur tiers. Si ces ressources deviennent indisponibles, sont modifiées de manière imprévue ou subissent une latence importante, l'affichage dynamique du chat et de l'image peut être affecté. De plus, l'envoi de messages via une requête GET encodée en base64, bien que conforme aux consignes, pose des limites en matière de sécurité et de robustesse, notamment face à des entrées utilisateur malformées ou à des usages abusifs.

Sur le plan des données, l'utilisation d'une base SQLite constitue également une limite. Ce type de base de données est adapté à un projet académique ou à de petits volumes de données, mais il n'est pas conçu pour des environnements multi-utilisateurs

intensifs. Des problèmes de verrouillage ou de performances pourraient apparaître si le volume de messages ou d'agents augmente significativement.

Enfin, l'analyse statistique proposée dans l'application, bien qu'efficace pour visualiser certaines tendances, reste dépendante de requêtes exécutées. Dans le cas de volumes de données importants, ces calculs pourraient devenir coûteux en ressources et ralentir l'affichage des profils utilisateurs. Une solution plus robuste nécessiterait une pré-agrégation des données ou l'utilisation de tâches asynchrones.

En termes de développements futurs, plusieurs améliorations pourraient être envisagées si davantage de temps était disponible. L'adoption d'une architecture plus robuste, incluant un serveur de production, une base de données relationnelle plus performante et une gestion asynchrone des requêtes, permettrait d'améliorer la fiabilité globale du système. De même, l'intégration d'un mécanisme d'authentification renforcé pour les échanges externes, ainsi qu'une meilleure gestion des erreurs liées aux services tiers, renforcerait la sécurité et la stabilité de l'application.

5.Conclusion

Pour conclure, une application Django orientée "agents IA + chat" a été développée et enrichie par étapes. Un système de conversation persistant a été implémenté, puis une IA locale via Ollama a été intégrée afin de produire des réponses automatiques sans service externe. Des statistiques ont été ajoutées directement dans la page profil, afin d'offrir une visibilité sur l'utilisation réelle du système. Enfin, l'extension HELBPlays a été intégrée, permettant l'affichage dynamique de ressources externes et l'envoi de messages au chat collaboratif.

Malgré ces résultats, certaines limites restent présentes, notamment la qualité variable d'un petit modèle local, et l'absence d'un vrai temps réel côté HELBPlays (polling). Ces limites ouvrent des pistes d'amélioration concrètes : amélioration de la robustesse IA (modèle plus grand ou "API model"), application stricte du cooldown côté serveur, et durcissement de la cohérence des conversations. Globalement, le projet présente une base fonctionnelle solide, et les choix techniques réalisés restent cohérents avec les objectifs pédagogiques du cours.

6.Vidéo

https://youtu.be/nPycI7PvtXk